



UNIVERSITAT
ROVIRA I VIRGILI



UNIVERSITAT DE
BARCELONA



UNIVERSITAT POLITÈCNICA
DE CATALUNYA
BARCELONATECH

DEEP REINFORCEMENT LEARNING-BASED JOINT HANDOVER AND TRAJECTORY OPTIMIZATION FOR 5G-CONNECTED UAV

MARINA BERMÚDEZ GRANADOS

Thesis supervisor

SERGI ABADAL CAVALLÉ (Department of Computer Architecture)

Thesis co-supervisor

EVGENII VINOGRADOV (Department of Computer Architecture)

Degree

Master's Degree in Artificial Intelligence

Master's thesis

School of Engineering

Universitat Rovira i Virgili (URV)

Faculty of Mathematics

Universitat de Barcelona (UB)

Barcelona School of Informatics (FIB)

Universitat Politècnica de Catalunya (UPC) - BarcelonaTech

11/05/2026

Abstract

In recent years, there has been a notable rise in interest around autonomous aerial vehicles. In spite of their broad applications and their affordable deployments, these drones still require a reliable connection in order to successfully accomplish their assignments. Since private networks are expensive and unscalable in urban scenarios, the best alternatives rest with 5G and 6G cellular networks. Still, the unique experiences of the UAVs at high-altitude speeds complicate their handover management. Most research addresses trajectory planning and handover management as separate problems, often incorporating abstract environments in the process. In this thesis, our objective is to jointly optimize both the itinerary and the handover schemes of a UAV. Through deterministic raytracing simulations in a realistic 3D model of Barcelona, we design a realistic environment for an RL agent. After extensive finetuning and experimentation, we have determined that our model is capable of optimising its metrics with a few path-finding mistakes along the way. In other words, our solution gives priority to persistent connectivity rather than task completion.

Keywords— Unmanned Aerial Vehicle (UAV), Joint Trajectory and Handover Optimization, Trajectory Design, Handover Management, Deep Reinforcement Learning (DRL), Proximal Policy Optimization (PPO).

GitHub Project— <https://github.com/Nara-On/JointHandoverTrajectoryOptimization>

Contents

1. Introduction	2
2. Related Work	2
2.1. The role of Artificial Intelligence in 6G	2
2.2. Handover Optimization	3
2.3. Trajectory Optimization	4
2.4. Joint Optimization	5
3. System Model	5
3.1. Channel Description	6
3.1.1. Realistic Urban Environment and Network Topology	6
3.1.2. Ray-Tracing Simulations and RSS Computation	6
3.2. UAV Movement	9
3.3. Metrics Simulation	9
3.3.1. Handovers and RBG Allocation	9
3.3.2. Transmission Delay	9
3.3.3. Interference	10
3.3.4. Energy Consumption	10
4. Problem Formulation	10
5. Proposed Solution	11
5.1. Background: Reinforcement Learning	11
5.2. Environment Design	12
5.3. Proximal Policy Optimization (PPO)	13
5.4. Greedy Benchmark	13
6. Performance Evaluation	14
6.1. Experimental setup	14
6.2. Optimizing PPO	15
6.3. Configurations	16
7. Analysis and Results	16
7.1. Comparative of PPO with the Greedy Benchmark	17
7.2. Comparative between Different Policies	19
7.3. Practical Insights	19
8. Conclusions and Future Work	21
A. List of Abbreviations	25
B. Handover Visualizations	26

1. Introduction

The Low-Altitude Economy (LAE) has been gaining considerable traction over the past few years with its promise of low-cost, highly flexible transportation. This emerging industry encompasses the aircraft vehicles themselves along with all the necessary infrastructure to support their deployment in a myriad of assignments. From logistics and infrastructure inspection to surveillance, these drones require reliable wireless connectivity for mission updates, fleet coordination, and occasional high-bandwidth up-link transmissions. While proprietary aviation technologies and private networks offer these services to some extent, they are expensive and lack the scalability required for dense urban operations. Moreover, traditional users of these technologies express concerns of increasing interference to their services. Consequently, 5G and emerging 6G cellular networks are the most viable infrastructure for large-scale Unmanned Aerial Vehicle (UAV) integration [1].

However, current terrestrial networks are optimized for 2D ground User Equipment (UE), typically utilizing down-tilted antennas [2]. As users of the network, UAVs experience unique radio environments characterized by Line-of-Sight (LoS) links to multiple Base Stations (BSs) and signal nulls due to the antenna radiation patterns, as well as sudden LoS blockages. The inherent high mobility experience of UAVs, combined with these fluctuating signal conditions, makes Handover (HO) management a critical challenge [3]. Frequent or poorly timed handovers result in increased signaling overhead, packet loss, delays, and potential link failures, jeopardizing autonomous mission safety and efficiency.

Existing research often treats trajectory planning and handover optimization as decoupled problems or relies on abstract simulations that neglect the nature of the aerial communication channel and standardized networking protocols. This thesis addresses these gaps by proposing a joint optimization framework for UAV trajectory and handover management using Deep Reinforcement Learning (DRL). Unlike existing models, our approach integrates deterministic raytracing simulations in a realistic 3D model of Barcelona with real locations of Base Stations, 3GPP-compliant handover triggers, and realistic resource allocation. By co-optimizing the flight path and connectivity parameters, we aim to maximize up-link reliability and minimize redundant handovers in a realistic cellular-connected UAV environment. The performance evaluations will reveal that our approach is capable of optimizing its metrics despite some difficulties in navigation. Our proposed solution emphasizes absolute connection to the network rather than task completion.

The remainder of this paper will describe the current state of the art in Section 2 and our system model in Section 3. Then, we will detail the problem formulation for our UAV agent in Section 4, plus our proposed solution in Section 5 for both our realistic environment and our deep RL model. From there, we will describe the relevant details of our experiments in Section 6 along with the consequent results and insights in Section 7. The final conclusions and future research avenues will be reflected in Section 8. In the appendix, we provide a complete list of all the abbreviations applied in this work, specifically in Table 5, as well as some extra materials.

2. Related Work

Several studies tackle the potential improvements of AI tools in 5G networks, including the optimization research of various metrics in different scenarios. In this section, we will define the state of the art in these techniques and how researchers handle UAV optimization. A summary is provided in Table 1 with all relevant details of every mentioned study.

2.1. The role of Artificial Intelligence in 6G

The upcoming sixth generation of mobile communications will require the support of many fields, including Artificial Intelligence (AI). Compared to traditional approaches, AI techniques can adapt to changing environments and learn from real-world experience [21]. The optimization of complex tasks and procedures in these communication networks can be quite challenging, but AI models offer complex task automation with tractable

Ref	Year	Study Focus	UAV Perspective	Solution	Simulation Details
[4]	2021	Handover	User	Q-learning	Not specified
[5]	2022	Handover	User	PPO	Unity 3D environment
[6]	2023	Handover	Provider	TOPSIS and Q-learning	Not specified
[7]	2025	Handover	User	DQN in XAI framework	Simulated and real-world data
[8]	2025	Handover	User	Bayesian optimization with transfer learning	3D model generated from OpenStreetMap data in London
[9]	2019	Multi-Agent Trajectory	Provider	Multi-agent Q-learning	Not specified
[10]	2020	Multi-Agent Trajectory	Provider	Multi-agent DDPG	Squared-shaped area
[11]	2021	Trajectory	Provider	DRL-based actor-critic networks	Rectangled-shaped area
[12]	2021	Trajectory	User	DRL with actor-critic networks and a pointer network A*	Rectangled-shaped area
[13]	2021	Trajectory	Provider	Q-learning as a MAB problem	Hexagonal layout and a central BS
[14]	2021	Trajectory	Provider	DQN	Squared-shape area
[15]	2021	Trajectory	Provider	DQN and a reward-based greedy algorithm	Squared-shape area
[16]	2023	Multi-Agent Trajectory	Provider	Independent Q-learning with DRQN	3D space
[17]	2023	Trajectory	User	Q-learning	Data from height maps extracted from a real data simulator
[18]	2023	Joint	User	D3QN	Urban rectangular area with 4 BS in the corners
[19]	2025	Joint	User	Minimum weight path algorithm over a weighted finite graph	Squared-shaped area
[20]	2025	Joint	User	D3QN in a QMIX framework	Rectangular urban area with 4 BS in the corners
Our work	2026	Joint	User	PPO	Raytracing simulation in a realistic 3D model of Barcelona

Table 1: Summary of Related Work

solutions and data-driven decision-making algorithms.

We can discern three kinds of methods inside AI and, more specifically, machine learning (ML): supervised learning (SL), unsupervised learning (UL), and reinforcement learning (RL) [22]. Every approach has its own benefits and problems depending on the problem formulation. On one hand, SL utilizes labeled data to infer new sets of outputs in the form of predictions of numbers or categories. Despite the potential for accurate predictions, SL requires a labeled ground truth, which may not be available depending on the problem. On the other hand, UL employs unlabeled data to unravel hidden patterns within the data. Even though there is no longer the need for processed inputs during training, the distribution of the data may not present clear and relevant patterns. By contrast, RL does not require a distribution of labeled or unlabeled data but an environment for an agent to learn an optimal policy. Even with its dynamic management of complex environments, it may struggle with large or continuous spaces.

All strategies detailed in Table 1 have already been applied to Non-Terrestrial Networks (NTN), from drone-based connectivity and resource allocation to anomaly detection and routing. However, the designated AI technique must be suitable to the corresponding tasks. Since our objective is the optimization of trajectory, handovers, and connectivity metrics, this paper will employ RL for its focus on formulating an optimal policy for an independent agent. Note the fact that RL has also been employed in multi-agent instances [23]. Still, most implementations rely on abstract assumptions and simulated environments. We hope that our study in more realistic environments for an independent UAV agent will facilitate future integration and implementation in real-world scenarios.

2.2. Handover Optimization

We can define a handover as the process of transferring a connection from one cell to another without disrupting service, with the objective of maintaining a continuous, reliable, and seamless reception [24]. These switches between cells can usually be determined through the measurement of various metrics, such as the Reference Signal Received Power (RSRP), the Received Signal Strength Indicator (RSSI) of the serving BS, or the Signal-to-Interference-plus-Noise Ratio (SINR) [25]. Once one of these factors reaches an undesirable threshold, a handover must be performed from the current cell to a more suitable one. Traditional protocols for handover decision-making tend not to extrapolate well to aerial devices due to their high mobility at soaring altitudes. Furthermore, the higher elevations of the drones unfold a larger amount of LoS links to non-serving BSs, producing larger interferences in the network. Given the drone's high-reaching speeds plus their increased LoS connection probabilities, drone networks are more vulnerable to the handover ping-pong effect, where a device performs unnecessary and redundant handoffs repeatedly. In consequence, the Quality of Service (QoS) greatly decreases.

An alternative handover optimization strategy is to employ AI tools in order to automatically perform handovers based on the current circumstances in the environment. Most research in handover decision-making can be

divided into two different perspectives: UAVs as users of the network or UAVs as providers of service. Nevertheless, the majority of handover management studies consider the UAVs as users.

A good number of studies employ Q-learning techniques, an off-policy branch of RL models designed to evaluate the quality of the taken actions. For instance, the paper [4] utilized a Q-learning-based approach to optimize the mobility of a user UAV in an ultra-dense network. There are even studies where Q-learning is combined with other methods like the Technique for Order of Preference by Similarity (TOPSIS) [6], to optimize the handover decision-making of a UAV provider in a heterogeneous network. More recent works are also integrating explainability frameworks into Deep Q-Network (DQN) models [7], improving our understanding in parameter sensitivity. Alongside Q-learning methods, there have been a few instances of policy-gradient algorithms devised to learn the most optimal handover policy. RL models such as Proximal Policy Optimization (PPO) [26] have been used to teach UAV users an effective handover scheme [5]. Beyond the usual RL models, some studies have ventured into different strategies for the hyperparameter optimization of A3-offset and Time To Trigger (TTT) parameters. In the case of the study [8], the researchers applied Bayesian optimization with transfer learning on a 3D model generated from OpenStreetMap data from an urban rectangular area in London.

2.3. Trajectory Optimization

The trajectory of a UAV has a clear effect on its energy consumption, its connectivity, and its handover rate. After all, its positioning in the space can widely alter its LoS links to non-serving BSs and, in turn, influence the quality of its reception. Not only that, but its travel speed will also impact its connectivity due to the expected amount of necessary handovers. The sensitivity of these factors has contributed to a large portion of UAV optimization research focusing on trajectory planning in particular. Moreover, most research is centred around the minimization of the drones' energy consumption since an optimal itinerary would not exhaust the UAVs' battery during a flight.

Similar to handover optimization, trajectory optimization studies also discern between UAVs as users of the network and UAVs as providers of reception. However, most studies are centered around arranging the drones in the space to procure an optimal wireless connection. Inside the exceptions to this pattern, we can find a myriad of suitable strategies. For instance, the study [17] explores the minimization of handovers, disconnectivity, and energy consumption of cargo-UAVs with a Q-learning-based methodology. The researchers in paper [12] also examine the minimization of the energy consumption for a data collection drone, but through the lenses of a sequence-to-sequence prediction problem.

Moving to the service provider drones, the usual methodology consists of applying RL models to a simulation of one or multiple UAVs. As an illustration, researchers in [14] design the trajectory of a drone in order to serve mobile ground users employing a DQN-based learning model. Furthermore, the study in [13] also applied a Q-learning-based approach to design the trajectory of a UAV provider, with a unique formulation as a multi-armed bandit (MAB) problem. Some papers go beyond the usual trajectory planning methods by integrating deep learning (DL) into their solutions. For instance, a DRL-based algorithm is used in [11] with an actor-critic architecture to optimize the flight patterns of a drone in a rectangular area. There are even comparative studies such as the paper in [15], where a DQN and a reward-based greedy algorithm are implemented for the optimization of UAV-mounted mobile BSs.

There is a major complexity difference going from one agent to multiple ones. Most papers dealing with a multi-agent system rely on Q-learning algorithms, such as the Deep Deterministic Policy Gradient (DDPG) [10]. A similar solution was achieved in [9] through the management of route and power consumption in the wireless connection of multiple agents. Even so, the added complexity may require another step of intricacy. In [16], the trajectory design of multiple UAVs is implemented with independent Q-learning and Deep Recurrent Q-Networks (DRQN) in order to provide service to a given area.

2.4. Joint Optimization

In recent years, there has been a surge of studies attempting to jointly optimize the trajectory and the handovers of UAVs in a simultaneous manner. The objective is to yield finer results than the sum of its parts. In papers [18], [19], researchers design a joint problem formulation for the optimization of various metrics. In particular, they employ a Dueling Double Deep Q-Network (D3QN) model or an algorithm capable of finding the minimum weighted path from a weighted finite graph. There are even multi-agent implementations that transform the original D3QN model with a QMIX framework [20].

Despite the successes of these approaches, there are still a few setbacks to their strategies. For instance, the action spaces tend to be severely condensed so the problem may become tractable. Most papers perform their simulations in simplified maps with a limited number of BSs in unrealistic placements or movement directions. An agent with limited mobility or a working area with simplified characteristics will inevitably affect the eventual realism of the simulations. The closing of the sim-to-real gap is already quite a complex problem in nature, but the excessive reliance on abstractions will not improve it either. As more dimensions are added into the space, the complexity of the model needs to escalate too. The simplified environments can be handled with Q-learning methods such as DQN or DDPG. Once the environments reach larger dimensions, the problem starts demanding more suitable methodologies. Hence, we take these shortcomings as the motivation for our work. Our objective is to design a realistic environment along a relevant RL model. In the following sections, we will describe the design and implementation of all the relevant aspects of our RL model.

3. System Model

In Figure 1, we examine the setup for our system model: a UAV agent traversing a rectangular map filled with BSs and other UE. Given a starting position and an end goal, the UAV must find the most optimal trajectory so as to not exhaust its energy nor provoke redundant handover operations. During its trajectory to the given goal, the agent will be able to transfer its current BS connection to another BS which is not currently serving a UAV. Every BS will have a certain number of Resource Block Groups (RBGs) which may be occupied by the UAV or other terrestrial users so as to imitate real-world situations.

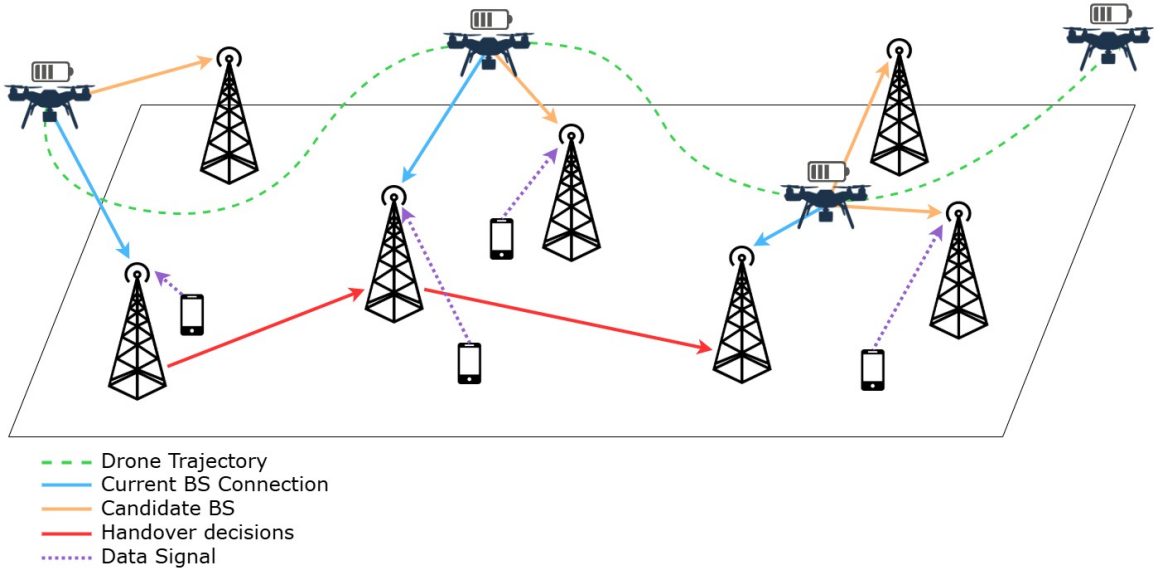


Figure 1: System Model

3.1. Channel Description

The simulation of channel conditions is an essential component to a realistic environment, from the distribution of BSs in the area to the consequent connectivity computations. Hence, we have extracted a real-world distribution with the objective of simulating accurate channel conditions. We will examine all the relevant implementation details within this section.

3.1.1. Realistic Urban Environment and Network Topology

To evaluate the proposed 5G UAV handover mechanism, a high-fidelity spatial map of cellular coverage was constructed for the urban environment of Barcelona. The 3D building geometry was sourced from the 3D-GloBFP dataset [27], providing footprint-scale building heights. This data was imported as a 3D Standard Tessellation Language (STL) representation to serve as the physical interaction layer for Radio Frequency (RF) propagation. The macro-cellular 5G network topology was mapped using real-world BSs coordinates and frequency band allocations from the Spanish Minetur Geoportal [28]. The network operates at a carrier frequency of 2.1 GHz (typical 5G NR n1 deployment) and is partitioned among three major network operators. As illustrated in Figure 2, the simulation comprises 133 BS sites partitioned among three network operators. Specifically, Operator 1 utilizes 87 sites, Operator 2 utilizes 59 sites, and Operator 3 utilizes 41 sites. A significant portion of the network (43 sites) consists of multi-tenant shared infrastructure. To ensure realistic aerial coverage modeling, all antennas were mounted on 3-meter masts atop the building roofs.

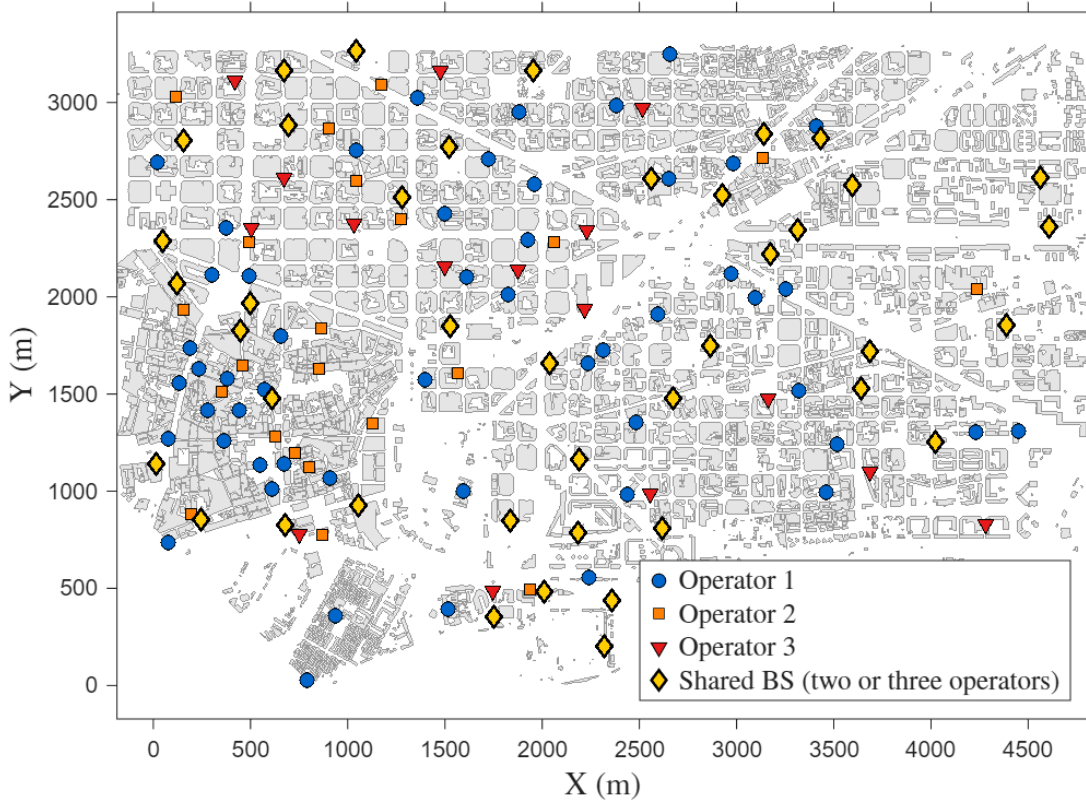


Figure 2: Spatial distribution of the 133 simulated Base Stations across the Barcelona urban grid. The map highlights dedicated sites for the three operators and 43 multi-tenant shared locations.

3.1.2. Ray-Tracing Simulations and RSS Computation

Unlike terrestrial UEs, UAVs cruising at 100 m altitudes experience high LoS probability but frequent sudden blockages. Consequently, popular empirical statistical models (e.g., Okumura-Hata) are insufficient [2]. Moreover, statistical models fail to capture spatial propagation effects [29] making them inapplicable for investigating the mobility-induced effects such as handovers. To accurately capture the multipath environment and the

specific impact of antenna sidelobes, spatial channel data was generated using a deterministic ray-tracing engine in MATLAB.

The simulation parameters are summarized in Table 2. The ray-tracer was configured to compute up to two maximum reflections. Diffraction was deliberately disabled; at a 100 m flight altitude, the propagation environment is heavily LoS-dominant, with primary signal degradation occurring via free-space path loss and rooftop/ground bounces. Eliminating diffraction significantly optimized computational overhead without sacrificing accuracy at the aerial UE altitude.

Each BS was configured with three sectors, simulated using a phased array featuring a 120° azimuth and 30° elevation beamwidth, all sectors have a downtilt of -12° . This standard downtilt is optimized for ground coverage, intentionally placing the 100 m altitude UAV in the upper sidelobes and nulls of the radiation pattern (a primary RF challenge [3] that the proposed handover algorithm must navigate).

Parameter	Value
Carrier Frequency	2.1 GHz (5G NR n1)
Transmit Power (P_{tx})	46 dBm
Antenna Model	Phased Array (120° Az, 30° El)
Downtilt	-12°
UAV Cruising Altitude	100 m
Total size	5000 m $\times$ 3500 m
Simulation Resolution	1 m
Number of Reflections	2
Total Base Stations	133

Table 2: Ray-Tracing and System Parameters

The final Received Signal Strength (RSS), expressed in dBm, was computed for every point in the spatial grid. The ray-tracer calculates the discrete path loss (PL) based on 3D geometry and material properties. The final RSS is derived using the standard downlink budget equation:

$$RSS = P_{tx} + G_{tx}(\theta, \phi) - PL, \quad (1)$$

where $G_{tx}(\theta, \phi)$ represents the 3D directional gain (in dBi) as a function of the azimuth (θ) and elevation (ϕ) angles between the transmitter and the UAV trajectory.

The resulting RSS values from all 133 base stations were aggregated into a comprehensive spatial database. Given the 5000×3500 m simulation area at a 1 m resolution, the ray-tracing engine computed 17.5 million spatial data points per sector of each base station (having 3 sectors). Note that the final dataset contained only values for the strongest sector of each BS. The resulting site-specific dataset serves as the foundational physical layer environment, enabling the deterministic evaluation of the proposed UAV handover algorithm as the aerial vehicle navigates the complex, interference-limited urban airspace.

To visualize the computed physical layer environment, Figure 3 illustrates the resulting spatial RSS distribution for a single transmitting base station, highlighting the complex propagation and shadowing patterns at the UAV's cruising altitude. By evaluating the maximum RSS across the aggregated database, the network serving cell map (handover boundaries) can be derived. Figure 4 depicts this assignment map for Operator 1, illustrating the fragmented coverage zones the aerial vehicle must traverse. Ultimately, this site-specific dataset serves as the foundational environment, enabling the deterministic evaluation of the proposed UAV handover algorithm within a complex, interference-limited urban airspace.

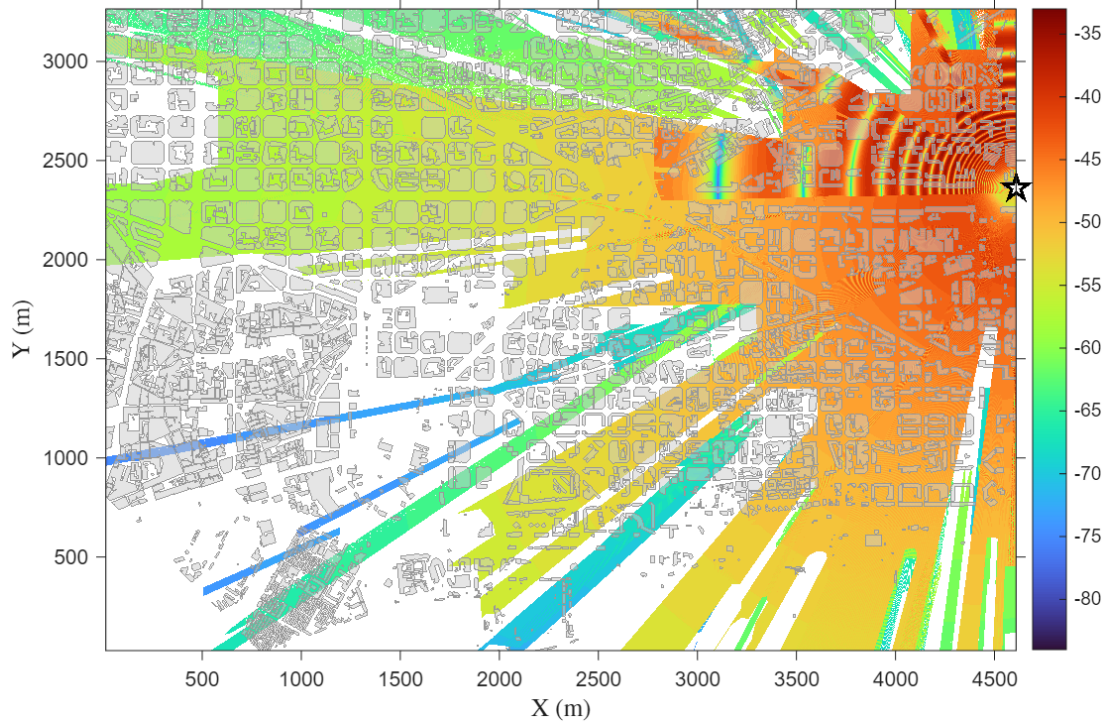


Figure 3: Spatial Received Signal Strength (RSS) distribution for a single base station at a 100 m altitude, illustrating propagation characteristics and building-induced shadowing.

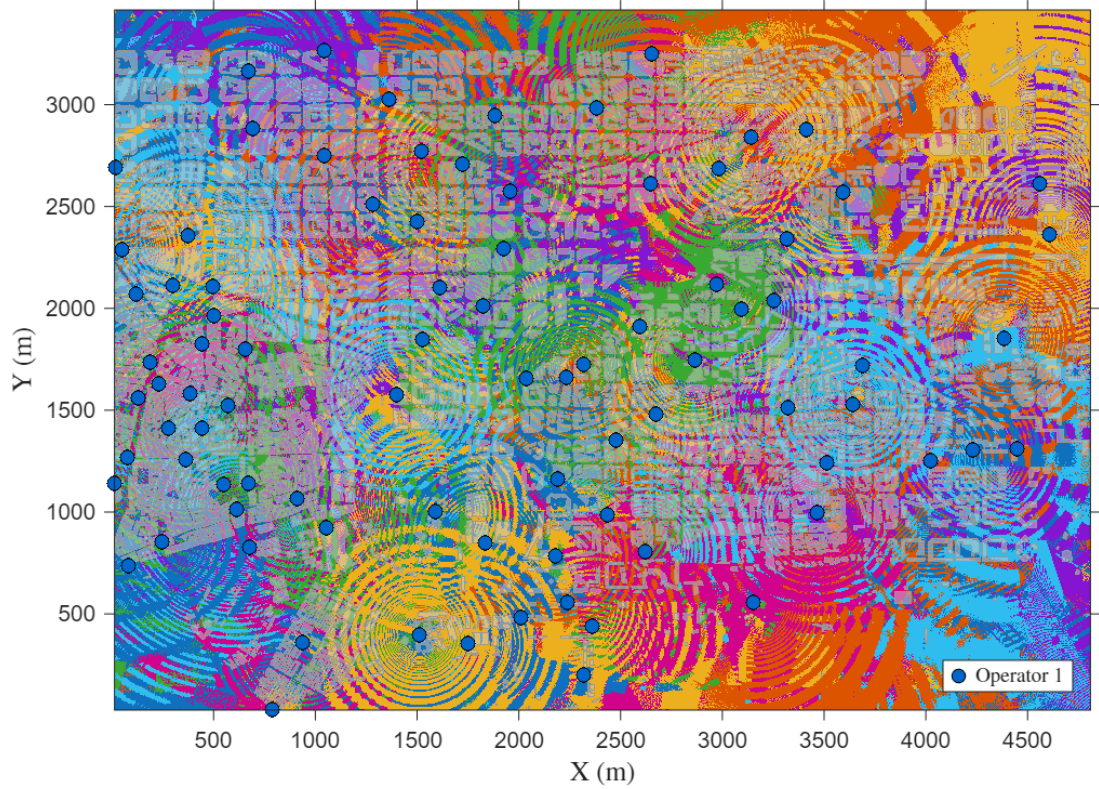


Figure 4: Network serving cell map (handover boundaries) for Operator 1 at a 100 m altitude. The colored regions indicate the dominant serving cell based on maximum RSS.

3.2. UAV Movement

The movement of the UAV will be achieved through its management of the acceleration and angular directions. Hence, the position can be determined by the uniformly accelerated rectilinear motion in every timestep t , formulated as:

$$\begin{aligned} x(t) &= x(t-1) + (v \cdot \cos \phi \cdot |t|) + \left(\frac{1}{2} a \cdot \cos \phi \cdot |t|^2 \right) \\ y(t) &= y(t-1) + (v \cdot \sin \phi \cdot |t|) + \left(\frac{1}{2} a \cdot \sin \phi \cdot |t|^2 \right) \end{aligned} \quad (2)$$

Note that the expectation for the UAV is to start from an initial standing position, accelerate towards the target position, and decelerate to reach a halt at the goal coordinates. Moreover, the drone will stay at a constant high altitude.

3.3. Metrics Simulation

Once the UAV is able to traverse across our map, we need to simulate the different actions and consequences it will face. The objective is for the agent to learn how to optimize its connection and itinerary through the different metrics of the system. In consequence, we ought to define the different responses of the model, including the requirements of a successful handover, the consequent quality of the connection, and the simultaneous energy usage.

3.3.1. Handovers and RBG Allocation

To transfer the connection from a current BS to a more suitable one, the UAV must formulate a handover request. The final result of this request will vary depending on the particular handover event at play. An event is a predefined set of measurement mechanisms and protocols proposed by 3GPP to perform a successful handover between two BSs. There are a myriad of handover events to choose from, but we have opted for a standard A3 handover [30]. Regardless of the drone's requests, an A3 handover will only be triggered if the metric items in the serving BS become greater than the metric items in the requested BS by a fixed hysteresis margin. Since our simulations are based on RSS values, our A3 handover events will compare the serving RSS_{k_s} against the requested RSS_{k_r} by the fixed hysteresis margin Δ :

$$h(t) = (RSS_{k_r}(t) > RSS_{k_s}(t) + \Delta) \quad (3)$$

Besides the BS requests, the drone could also request a specific RBG. The allocation of groups is determined entirely by their availability. This means that an RBG allocation will only be successful when no UE is occupying that particular group. If the request cannot be granted, the agent will just keep its current RBG. In the case of a simultaneous handover plus an unsuccessful RBG request, the new BS will allocate the first available group.

3.3.2. Transmission Delay

The resulting RSS value has a direct effect on the drone's ability to perform its designated tasks. When delivering a package or scanning an area, for illustrative purposes, the UAV would have to transmit its data to the serving BS. A suboptimal connection would cause the data to accumulate in its buffer, leading to possible dumps of important information. Hence, it is essential to manage the transmission delay $D(t)$ of the drone, given by the stored data inside the buffer $q(t)$ and the transmission rate $r(t)$:

$$D(t) = \frac{q(t)}{r(t)} \quad (4)$$

We will start with the stored data of the UAV. The current state of the buffer at timestep t is computed by adding all the received data packets and subtracting the processed ones, modeled as:

$$q(t) = q(t-1) + p(M = n|\lambda)MF_d + h(t)F_c - r(t)|t| \quad (5)$$

The drone will receive either general data packets of F_d bits or control packets of F_c bits. The number of packets received at any instant will vary depending entirely on its type. On the one hand, the amount of data packets will be generated through the Poisson probability:

$$p(M = k|\lambda) = \frac{\lambda^k e^{-\lambda}}{k!} \quad (6)$$

On the other hand, the control packets will only occur along a handover event $h(t)$. While these packets are being received, the UAV has to transmit all that information to an antenna. The transmission rate $r(t)$ is given by the bandwidth of an RRB W_B times how many RRBs there are per group $|B_G|$, plus the base two logarithm of the Signal-to-Noise Ratio (SNR) $\Gamma(t)$ at timestep t :

$$r(t) = (W_B \cdot |B_{G_k}|) + \log_2 \Gamma(t) \quad (7)$$

The SNR formula is quite simple in our case, since we only ought to subtract the RSS values of the serving BS $RSS_{k_s}(t)$ from the noise in the system. In our case, we will consider the thermal noise N_t along a constant noise figure N_f .

$$\Gamma(t) = RSS_{k_s}(t) - N_t - N_f \quad (8)$$

3.3.3. Interference

Besides the transmission delay $D(t)$, another concern in UAV optimization is the ensuing up-link interference. We may calculate the up-link interference by summing together all the RSS values from all neighbouring BS:

$$\mathcal{I}(t) = \sum_{n=1, n \neq s}^N RSS_{k_n}(t), \quad (9)$$

where N is the total amount of BS in the area and s is the current serving BS.

3.3.4. Energy Consumption

The last aspect to simulate in our system is the remaining energy of the UAV at every timestep t . The power consumption of an UAV [31] is given by:

$$E(t) = E(t-1) - \frac{m \cdot v}{370\eta r} + p \quad (10)$$

where m is the total mass of the UAV in kg, v is the cruising velocity, η is the power transfer, r is the lift-to-drag ratio, and p is the power consumption of its electronics. Most variables can be considered as constants in the simulation except the velocity of the drone. Since the energy consumption is dependent on the velocity and the velocity is managed by the drone's acceleration, the actions of the agent will inevitably have a direct impact on its ability to endure the mission.

4. Problem Formulation

We will define the problem formulation in this section with all the metrics described in our system model. The UAV will be present at coordinates $\{x(t), y(t)\}$, while assuming a constant altitude of h . Given an initial position $\{x_i, y_i\}$ and a final position $\{x_f, y_f\}$, the drone must reach it within T timesteps. At every timestep $t \in \{1, 2, \dots, T\}$, the agent will be able to select a moving direction α , an acceleration value a , its BS request $k(t)$, and its RBG request $G_k(t)$. According to the metrics of our system, we can define the objective function as follows:

$$\omega_t = w_1 \cdot D(t) + w_2 \cdot \mathcal{I}(t) + w_3 \cdot h(t), \quad (11)$$

where $w_i \in [0, 1]$ for $i \in \{1, 2, 3\}$ represents the corresponding weighting factors. All the weights should amount to one to effectively balance the metrics, that is, $\sum_{i=1}^3 w_i = 1$. From this function, the optimization problem is formulated as:

$$\min_{(x(t), y(t), k(t), G_k(t), T)} \sum_{t=1}^T \omega_t \quad (12)$$

subject to constraints:

$$(C1) \quad x_{min} \leq x(t) \leq x_{max}, y_{min} \leq y(t) \leq y_{max}, \forall t$$

$$(C2) \quad RSS_{k_s}(t) \geq RSS_{min}, \forall t;$$

$$(C3) \quad |v(t)| \leq v_{max}, \forall t;$$

$$(C4) \quad 0 \leq q(t) \leq q_{max}, \forall t;$$

$$(C5) \quad \sum_{n=1}^N \mathbb{1}(n = k(t)) = 1, \forall t;$$

$$(C6) \quad \sum_{m=1}^M \mathbb{1}(m = G_k(t)) = 1, \forall t.$$

where RSS_{min} is the minimum received RSS threshold, q_{max} is the maximum amount of bits in the drone's buffer, and v_{max} is the maximum velocity of the drone. These constraints ought to limit the agent's efforts to optimize the objective function. First, **(C1)** restrains the UAV's movements within the limits of the map, while **(C2)** forces the agent to meet specific RSS standards. Moreover, the constraint **(C3)** limits the drone's speed within a safe range below v_{max} . The remaining constraints ensure the correct inner workings of the agent, such as **(C4)** assuring the buffer storage exists between 0 and a maximum buffer capacity q_{max} . Both **(C5)** and **(C6)** guarantee the unique connection of the drone to one BS and one RBG from that BS. Note that the function $\mathbb{1}$ turns one when the expression inside is true, and zero otherwise.

5. Proposed Solution

From the functions and constraints, we can design an effective solution for our drone. The first step is to define the state and action spaces of the environment, followed by the description of the reward function. Once we have the environment characteristics, we will detail the implementation of the deep RL model while considering possible improvements from the reviewed literature. For comparative purposes, we will also specify the greedy benchmark employed in future evaluations.

5.1. Background: Reinforcement Learning

Before detailing the design specifications, we offer some background knowledge about the inner workings of RL. The main objective is for an agent to learn how to behave in a given environment. In other words, the agent must learn a policy reflecting the best actions to take at any given moment. In a mathematical sense, the problem formulation resembles a Markov decision process (MDP) defined by $(\mathcal{S}, \mathcal{A}, \mathcal{P}_{sa}, \gamma, \mathcal{W})$. The states s in the state space $\mathcal{S}$ are composed by all the possible configurations in the environment that are relevant to the agent. In a similar way, the actions a in the action space $\mathcal{A}$ represent the different ways the agent can interact with the environment. Every action has a consequent state defined by the state transition probability $\mathcal{P}_{sa}$, plus a consequent evaluation given by the reward function $\mathcal{W}$. A discount factor γ is also usually included to reflect the importance of future rewards. From this MDP framework, the agent must learn the most suitable actions from any given state so as to maximize its reward function.

The RL agent must learn these policies through the recollection of experiences in an environment. We can define a RL environment as a simulation scheme determining the sets of allowed actions and states, plus the corresponding rewards for every state. During training, the agent will attempt to complete different episodes while updating its internal policy. An episode refers to an isolated execution of an environment, from start to

finish. The length of an episode is measured in timesteps, which also defines how many interactions the agent has in every episode. An episode will usually persist for a determined number of timesteps unless it happens to be truncated. A truncation occurs when the actions of the agent have led to an early termination of an episode, whether it is due to an unrecoverable mistake or a successful completion. Note that an episode could run for an infinite amount of timesteps, but it is not relevant to our particular use case.

5.2. Environment Design

The environment is composed of states and actions limiting the agent's interactions while learning a policy. Inside the state space $\mathcal{S}(t)$ at timestep t , we can find the coordinates $\{x(t), y(t)\}$, the serving BS $k_s(t)$, the allocated RBG $G_{k_s}(t)$, the UAV's current energy levels $E(t)$, and the state of its buffer $q(t)$. Hence, the state is defined by the vector $\mathcal{S}(t) = [x(t), y(t), k_s(t), G_{k_s}(t), E(t), q(t)]$. Given a state at timestep t , the agent must decide a new moving direction and acceleration, plus its requests for a BS $k_r(t)$ and corresponding RBG $G_{k_r}(t)$. The resulting action is formed as $\mathcal{A}(t) = [a, \phi, k_r(t), G_{k_r}(t)]$, taking into consideration that all values should be discretised. It is important in RL to consider the data type of the actions because the resulting number of possible combinations will affect the learning process. The connection requests are already discrete variables since they are represented through the identifications of the BSs and RBGs. However, the acceleration and direction values are continuous variables in nature, requiring direct segmentation into discrete bins.

As the agent generates actions from the states, the environment returns evaluation values in the form of rewards. The reward function is based entirely on the aforementioned objective function, including the metrics of transmission delay $D(t)$, up-link interference $\mathcal{I}(t)$, and handoffs $h(t)$. All metrics have different scales which must be compensated for through the balancing factors β_i , where $i \in \{1, 2, 3\}$:

$$\begin{aligned} R_D &= \frac{1}{1 + \beta_1 D(t)} \\ R_{\mathcal{I}} &= \frac{1}{1 + \beta_2 \mathcal{I}(t)} \\ R_h &= \frac{1}{1 + \beta_3 h(t)} \end{aligned} \quad (13)$$

These individual rewards must be combined with three penalties found along the constraints in the problem formulation. As a result, the reward function $R(t)$ is formulated as the weighted addition of multiple individual rewards and a few limiting penalties:

$$\begin{aligned} R(t) &= w_1 \cdot R_D + w_2 \cdot R_{\mathcal{I}} + w_3 \cdot R_h \\ &+ \mathbb{1}((x(t) \notin [x_{min}, x_{max}]) \vee (y(t) \notin [y_{min}, y_{max}])) \cdot \mathcal{P}_1 \\ &+ \mathbb{1}(RSS_{k_s}(t) \leq RSS_{min}) \cdot \mathcal{P}_2 \\ &+ \mathbb{1}(|v(t)| \geq v_{max}) \cdot \mathcal{P}_3 \\ &+ \mathbb{1}(d(t-1) > d(t)) \cdot \mathcal{B} \end{aligned} \quad (14)$$

The first penalty $\mathcal{P}_1$ punishes the agent for leaving the map grid, while the second one $\mathcal{P}_2$ ensures a minimal connection quality compliance. In a similar way, the third penalty $\mathcal{P}_3$ guarantees the safe navigation of the UAV through the space. The minimal connection threshold will prevent any dumping of important data packets, while the speed limitations will help the drone regulate better its energy reserves. Note the last term $\mathcal{B}$, a bonus value for whenever the agent approaches the goal coordinates. In other words, the agent will receive a bonus in timestep t if its distance to the goal is smaller than in the previous timestep $t-1$.

During the learning process of the agent, the UAV will commit many mistakes, including unrecoverable ones. In the case of our agent, we will consider the exhaustion of all its energy a critical error. Therefore, the running episode will finish prematurely when the agent exhausts all its energy reserves. In consequence, the reward system will be substituted by $R(t) = \mathcal{P}_4$, a fourth penalty designed to punish the UAV for this critical mistake. It is worth noting the lack of terminations once the agent has arrived at its goal. The reason behind this design decision is to teach the agent how to perform an effective brake. If we were to immediately terminate the episode

5.3. Proximal Policy Optimization (PPO)

The literature study in this work confirms the usefulness of RL models. Most papers employ Q-learning algorithms or value-based ones, depending on the approach. In particular, the precedents of joint optimization use D3QN [32], an uncommon combination of both Double DQN [33] and Dueling DQN [34]. Nonetheless, these models no longer fit the requirements of the problem formulation. A more realistic environment entails a larger action space and, in consequence, a methodology capable of dealing with all possible communications. That is precisely the reason why we have opted for Proximal Policy Optimization (PPO) [26]. Furthermore, PPO has demonstrated in [35] its effectiveness in closing the sim-to-real gap.

Our model PPO in pseudocode 1 is classified as an on-policy deep RL model. Its main philosophy is preventing large policy updates via an objective function clipped to limit changes around the current solution. This clipping operation prevents the policy from diverging or becoming unstable and therefore improving its performance. There are many implementations of PPO, but we have decided to employ the stable baselines library [36] for its compatibility with custom environments.

Algorithm 1 PPO-Clip

Input: Initial policy parameters θ_0 , initial value function parameters ϕ_0

- 1: **for** $k = 0, 1, 2, \dots$ **do**
- 2: Collect set of trajectories $\mathcal{D}_k = \{\tau_i\}$ by running policy $\pi_k = \pi(\theta_k)$ in the environment.
- 3: Compute rewards-to-go $\hat{R}_t$.
- 4: Compute advantage estimates, $\hat{A}_t$ (using any method of advantage estimation) based on the current value function V_{ϕ_k} .
- 5: Update the policy by maximizing the PPO-Clip objective:

$$\theta_{k+1} = \arg \max_{\theta} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^T \min \left(\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_k}(a_t | s_t)} A^{\pi_{\theta_k}}(s_t, a_t), g(\epsilon, A^{\pi_{\theta_k}}(s_t, a_t)) \right),$$

typically via stochastic gradient ascent with Adam.

- 6: Fit value function by regression on mean-squared error:

$$\phi_{k+1} = \arg \min_{\phi} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^T \left(V_{\phi}(s_t) - \hat{R}_t \right)^2,$$

typically via some gradient descent algorithm.

- 7: **end for**
-

5.4. Greedy Benchmark

Besides the RL models, we need a way to compare the performance of our approach with another algorithm to serve as a reference point. The usual solution is to implement a greedy strategy to compete with the more developed models. In our case, we will develop our own greedy strategy compatible with our realistic environment.

The first major difference is in the movement of the greedy agent itself. For simplification purposes, the UAV will maintain a constant velocity without any acceleration variations. Regarding the possible moving directions, the greedy agent will always select the closest discretised angle between its current position and the given goal. In other words, both greedy and PPO agents will have the exact same set of possible movement directions to ensure a fair comparison between the two.

The handover requests will be handled through the constant maintenance of time counters. For every BS in the map, the greedy benchmark will count how many consequent timesteps have taken place where a handover was viable. Once a counter arrives at a certain threshold, the handover will be performed to that particular BS,

and all the counters will be set to zero. If the handover to a BS becomes no longer feasible at any point before arriving at that threshold, the counter of that BS will be restarted. Moving to the RBG, the greedy drone will request the allocation of the first available group.

6. Performance Evaluation

Along with the design and implementation of our proposed solution, we should also assess its effectiveness in learning a productive policy. Thus, we will describe the different aspects related to the experiments, including the setup of our environment and all the configurations for the tests. In addition, we will also include the finetuning process of PPO for reproducibility purposes.

6.1. Experimental setup

Given the complexity of our realistic environment, this entire section will be dedicated to all the details surrounding its parameters. Although a summary will be provided in Table 3, we also offer an extended explanation on the design choices behind our complete experimental setup.

All experiments will be performed in our 5000×3500 map, where every timestep in the environment represents one millisecond within the real-world simulation. The service area has a distribution of 133 BSs employed by three operators: 87 for Operator 1, 59 for Operator 2, and 41 for Operator 3. All future experiments will employ the BSs belonging to Operator 1. Every BS will have 12 RBGs available along with 8 RRBs within each group. When the agent makes a handover request, the A3 offset will be set to 3 dBm in accordance with real A3 handover events. In a similar way, at least 50% of these RBGs will be randomly occupied by other UEs in order to simulate real-world conditions.

The agent's movement within the space is going to be limited to the eight main cardinal directions. In other words, the drone will be able to turn every 45° , towards north, south, east, and west, as well as north-east, north-west, south-east, and south-west. When it comes to its acceleration, the agent will be able to choose an acceleration value between intervals of 2.5 m/s^2 , from -5 m/s^2 to 5 m/s^2 .

Moving to the characteristics of the drone itself, the UAV will have a mass of 3 kg plus a lift-to-drag ratio of 5. Its buffer will have a capacity of 160 KB, while its battery will have a capacity of 1000 kW with a power efficiency of 55%. As for the inner electronics of the drone, they are set to consume 0.1 kW.

For the simulations within the buffer, we assume an average arrival of 100 data packets plus a constant of four control packets per handover event. Every data packet will occupy 1000 bits, while every control packet will take 1200 bits. The consequent transmission rate to process all this information is computed with a bandwidth of 180 kHz per RRB, as well as a constant noise figure of 9 dB. When it comes to the remaining thermal noise, the implementation is designed to vary from an initial value depending on the total bandwidth in the simulation. Given an initial value of -174 dBm and the total bandwidth, the thermal noise with the described parameters would be -112.41 dBm. Note that the total bandwidth is calculated as the multiplication of the bandwidth per RRB times the total amount of RRBs in an RBG.

The learning process is managed with the balancing coefficients, the weighting factors, the penalties, and the bonus terms. Starting with the balancing coefficients, their objective is to ensure all reward components are stabilised around the same dimensions. The final values can be examined in Table 3, along with the respective weighting factors for different policies. The next parameters are the penalties. All of them are set to -10, the second penalty will activate upon reaching an RSS of -96 dBm while the third penalty will be applied when the drone surpasses 100 m/s. As a bonus term for approaching the goal, we will reward the agent with an additional 12 points for every timestep closer to its objective.

Parameters	Labels	Values
Length of a timestep	$ t $	0.1 seconds
Service area	$x_f \times y_f$	5000×3500
Within goal threshold	δ	10 meters
Available BSs Operator 1	$ k_1 $	87 BSs
Available BSs Operator 2	$ k_2 $	59 BSs
Available BSs Operator 3	$ k_3 $	41 BSs
Available RBGs in BS	$ G_k $	12 RBGs
Available RRBs per RBG	$ B_{G_k} $	8 RRBs
A3-offset	Δ	3 dBm
UE Occupancy	-	50%
Acceleration range and discretization bins	a	$[-5.0, 5.0]$ m/s ² discretized to 5 bins
Angle range and discretization bins	ϕ	$[0.0, 2\pi]$ radians discretized to 8 bins
Mass	m	3.0 kg
Lift-to-drag ratio	r	5.0
Power transfer efficiency	η	55%
Power of electronics	p	0.1 kW
Maximum energy capacity	E_{max}	1000 kW
Maximum buffer capacity	q_{max}	160 KB
Average data packets received	λ	100
Size of data packets	F_d	2000 bits
Number and size of control packets	F_c	4 packets of 1200 bits
Bandwidth per RRB	W_B	180 kHz
Thermal noise	N_t	-112.41 dBm
Noise figure	N_f	9 dB
Balancing coefficients	β_s	$\beta_1 = 10, \beta_2 = 10^5, \beta_3 = 10^2$
Weighting factors for an equal policy	w_s	$w_1 = w_3 = 0.35, w_2 = 0.3$
Weighting factors for a transmission delay policy	w_s	$w_1 = 0.8, w_2 = w_3 = 0.2$
Weighting factors for an interference policy	w_s	$w_2 = 0.8, w_1 = w_3 = 0.2$
Weighting factors for a handovers policy	w_s	$w_3 = 0.8, w_1 = w_2 = 0.2$
Penalty parameters	$\mathcal{P}_s$	$\mathcal{P}_1 = \mathcal{P}_2 = \mathcal{P}_3 = \mathcal{P}_4 = -10$
Minimum RSS threshold	RSS_{min}	-96 dBm
Maximum speed threshold	v_{max}	100 m/s
Bonus closer distance	$\mathcal{B}$	12

Table 3: Simulation Parameters

6.2. Optimizing PPO

The PPO model is known for its learning stability and reliable performance due to its prevention of large updates through a clipped surrogate objective function. However, this learning effectiveness has to be adjusted to every problem formulation. The hyperparameters are an essential aspect for the correct workings of PPO. From the summary provided in Table 4, we will explore the different parameterizations of our model.

One of the most important parameters in any DL implementation is the learning rate α , batch size, and discount factor γ of a model. The learning rate represents the update rate of the model's parameters during training, whereas the batch size determines the amount of samples processed before the updates of the model weights. A larger learning rate can accelerate learning along with the learning instability, while a larger batch size can improve training stability at the cost of more memory requirements. The discount factor manages the degree of anticipation for future rewards; higher values motivate the agent to value them more.

Beyond the common parameters found in most algorithms, PPO incorporates a few novel functionalities within its implementation, like its clipped objective function. The clipping parameter ϵ manages the threshold for

clipping the ratio of policy probabilities. There are PPO variants which also include a Kullback–Leibler (KL) divergence term as a penalty coefficient. Another incorporation is a Generalized Advantage Estimation (GAE) lambda [37] in order to balance between bias and variance in the estimation of the advantage. There are two other balancing parameters in PPO: the value function coefficient c_2 balancing the value loss and policy loss in the computations of the total loss, and the entropy coefficient c_2 encourages exploration.

In the stable baselines framework, the PPO implementation utilizes a unique attribute that should be considered too. The parameter known as the number of steps determines the number of steps per environment before an update. In consequence, the number of updates occurring in an environment will depend on the total number of timesteps to perform per episode.

All these parameters must be properly finetuned in order to obtain the most suitable parameter combinations for our problem formulation and our realistic environment. After extensive testing, we have arrived at the final set of parameters provided in Table 4. During our experiments, we have been able to evaluate the effects of some particular attributes in the agent. For instance, lower learning rates were able to approach the goal more consistently than higher ones. We also noticed that lower batch sizes had consistently improved performances.

Parameters	Labels	Values
Number of episodes	-	300
Number of timesteps	-	2000
Number of steps	-	8
Learning rate	α	0.00003
Batch size	-	32
Gamma	γ	0.99
GAE lambda	λ_{GAE}	0.95
Clip range	ϵ	0.2
Value function coefficient	c_1	0.5
Entropy coefficient	c_2	0.01
Target KL	-	0.03

Table 4: PPO Parameters

6.3. Configurations

To test the effectiveness of our approach, we will be performing two sets of experiments. On the one hand, we will compare our PPO model with the greedy benchmark in an effort to set a reference point against other methodologies. On the other hand, we will explore if our RL model is capable of prioritizing particular metrics by examining different variations in the reward weighting factors.

All experiments will utilize the exact same set of parameters for the simulations, as well as the same set of hyperparameters for our PPO model. However, every execution will have its own particular set of reward weights to adapt the model according to the current circumstances. The comparisons with the greedy benchmark will employ the weights for an equal policy in Table 3. Meanwhile, the exploration into the model's prioritization capabilities will take the policies corresponding to the optimized metrics.

7. Analysis and Results

After designing our realistic environment along with our RL model, we have performed the experiments described in the previous section to evaluate the effectiveness of our system. In this section, we describe our results and the insights we can gather from them. Every set of experiments will measure the SNR values, outage time, up-link interference, remaining energy at the end of an episode, and number of handovers performed

during its executions. The SNR and outage durations measure the quality of the UAV's connection, whereas the up-link interference quantifies the amount of disruptions caused in the network. Since the energy efficiency (EE) is critical for a successful flight, we also ought to examine the leftover power in the drone once the episode has terminated. The last metric to capture is the number of handovers performed in order to monitor possible ping-pong effects. In addition to all these metrics, we will also examine the trajectory patterns of the drones.

7.1. Comparative of PPO with the Greedy Benchmark

The first set of experiments compares our PPO model with an equivalent greedy benchmark. An example of a trajectory can be examined in Figure 5, while the corresponding metrics are provided in Figure 6. We can clearly see in the itineraries of the drones the inner priorities of each approach. The greedy benchmark prioritizes reaching the goal as quickly as possible, while PPO seems to prioritize its metrics regardless of the final destination. This is a pattern that will become more noticeable in the following sections.

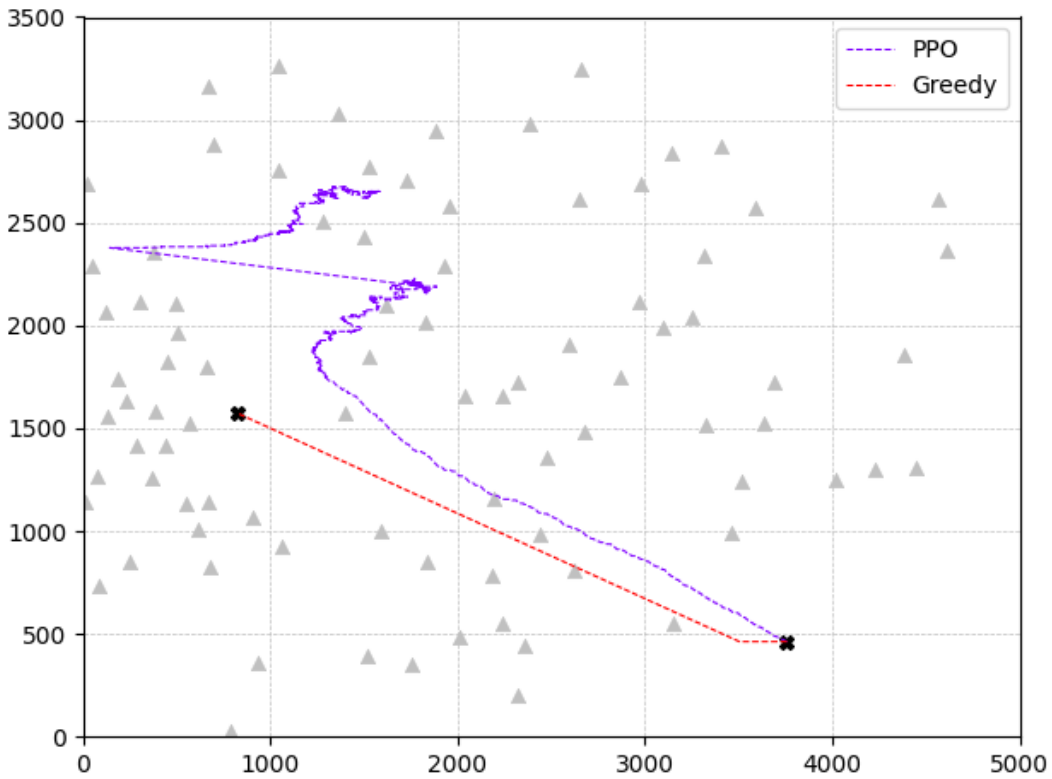


Figure 5: Trajectory comparison between PPO and greedy benchmark

Both models tend to demonstrate a similar performance in their resulting SNR values; they are quite similar in their leftover energies too. Since the energy consumption of the agent is directly related to its speed, we can confirm that PPO is adequately managing its acceleration in a safely manner. If that were not to be the case, the drone would constantly exhaust all its energy before the episode's termination. When it comes to outage times, both models have shown to occasionally produce a few seconds of unreliable connection. The main differences between the greedy benchmark and the PPO model occur in their up-link interferences and handover numbers. Our PPO model has the tendency of obtaining lower interference values, as well as fewer redundant handovers. We will provide qualitative examples about these handoffs in section B of the appendix, within Figure 9 and Figure 10 respectively.

Despite the irregularities in the trajectories of the PPO model, we must highlight its performance in Figure 6. The optimizations of our framework have successfully matched the greedy benchmark in all metrics, even surpassing it in interference and number of handovers. Our approach is giving more value to the optimization

of its metrics over the optimization of its trajectories.

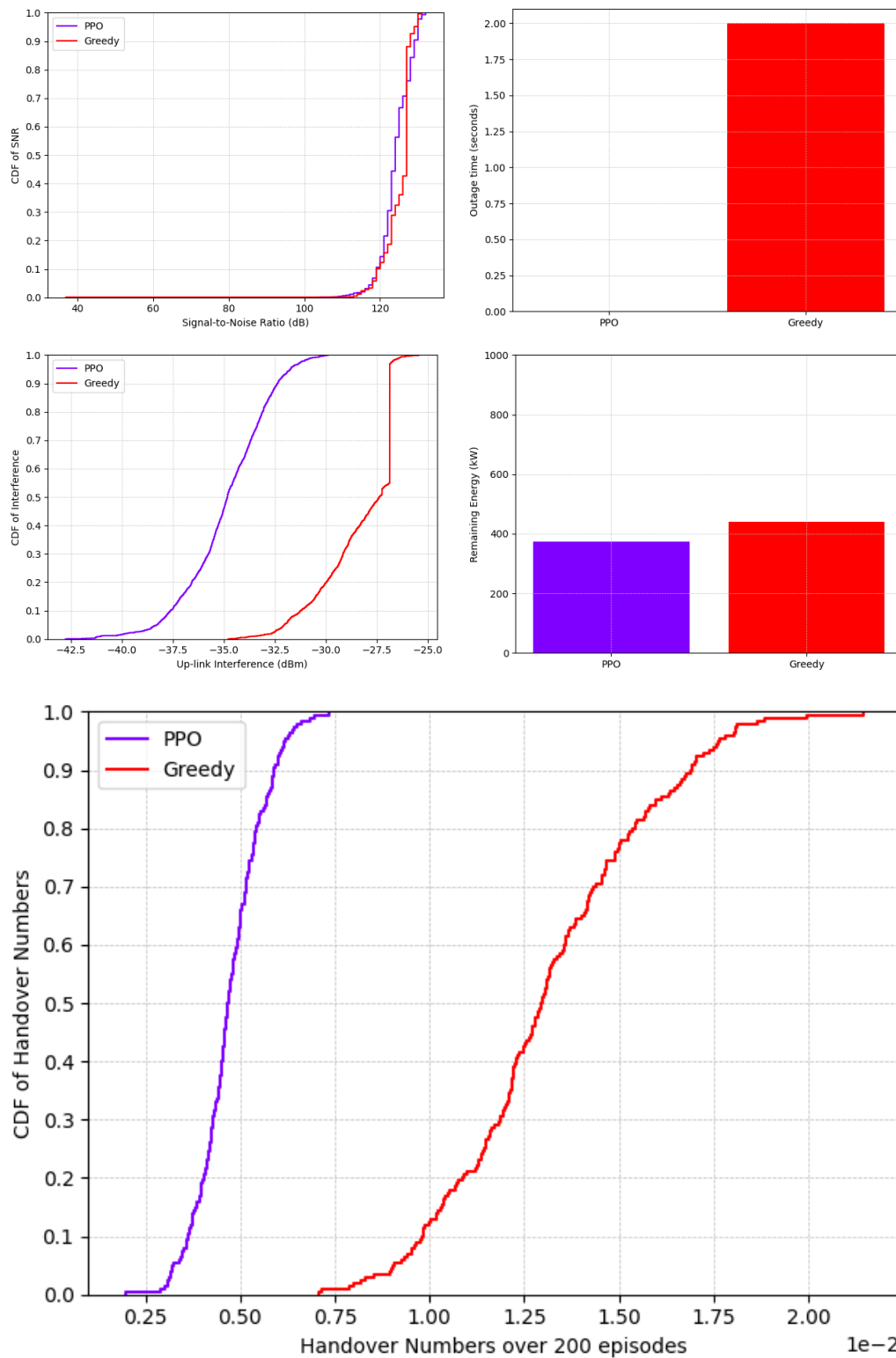


Figure 6: Comparisons of SNR, outage time, up-link interference, remaining energy, and number of handovers between PPO and greedy benchmark, from top left to bottom right

7.2. Comparative between Different Policies

This second set of experiments attempts to measure the performance differences when varying the given reward weights. The resulting trajectories can be found in Figure 7, along with their respective metrics in Figure 8. As aforementioned, PPO seems to have difficulties in managing its movement through the space. A clear example is provided within the handover policy, where the drone passes its destination and has to course-correct before approaching the goal again. Likewise, the transmission delay policy also seems to have an issue with its movement through the space. In contrast, the interference policy demonstrates an intention of approaching the goal by avoiding the central part of the map. The origin of these complications may occur in the formulation of the reward function or the movement implementation itself. In other words, either the agent does not have enough motivation to reach the goal or PPO requires more extensive training so as to obtain a good command on the movement controls.

On the metrics side of the analysis, we notice all three policies obtaining quite similar results. The occasional seconds in outage conditions, the remaining energy measurements, and the SNR values do not show any relevant disparities. However, there are slight differences in their up-link interferences and handover numbers. The transmission delay policy tends to obtain slim improvements compared to the other policies, with lower interference values and a lower number of handovers. As close seconds, the interference policy demonstrates a comparable performance in uplink interference, while the handover policy does the same in handover numbers.

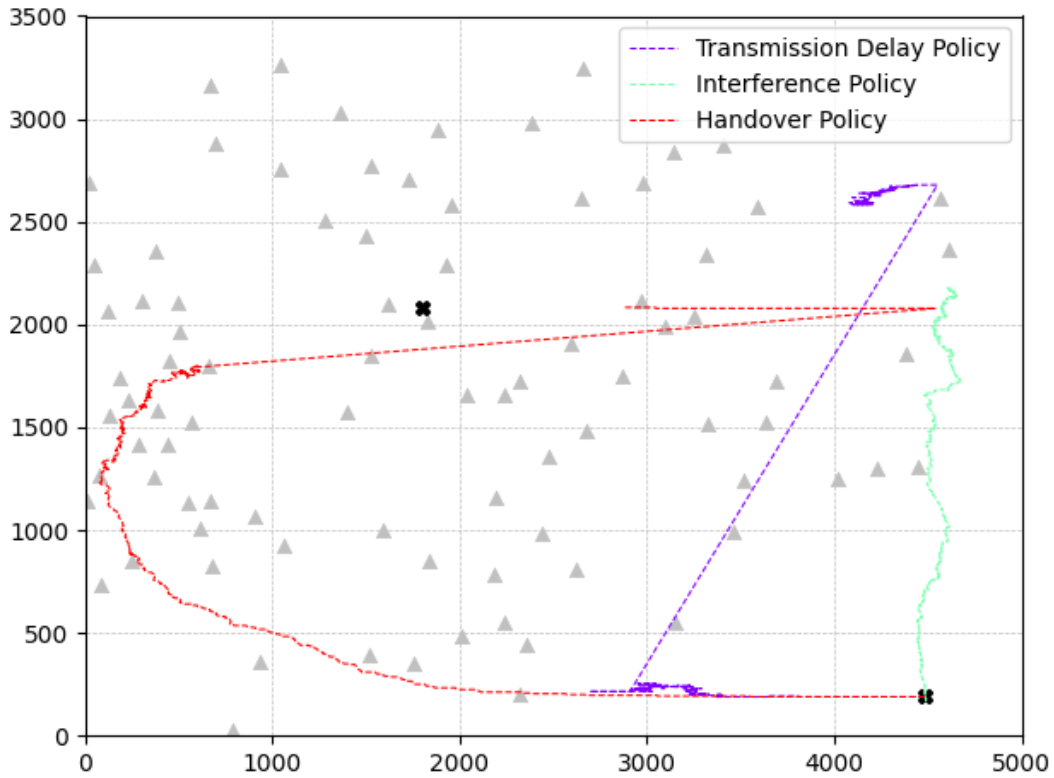


Figure 7: Trajectory comparison between our policies

7.3. Practical Insights

The results obtained in this section serve as a reflection of the many challenges associated with working with RL in realistic environments. From the balancing of reward factors to the consequent finetuning of all related parameters, the inherent intricacies within the environment become harder to learn for any RL agent. In spite of these challenges, we have implemented a model capable of optimizing its metrics even with some path-finding mistakes. Further improvements can be implemented in the future, the same way we have been implementing

many improvements during development of the project itself.

Before the experiments, there used to be a fourth weighting factor in the rewards function for the energy consumption of the agent, along with a specialized penalty for the data dumps caused by the overfilling of data packets in the buffer. We realized during testing that our formulations were not only redundant but greatly unbalanced as well. As aforementioned, the energy levels of the drone were already managed through

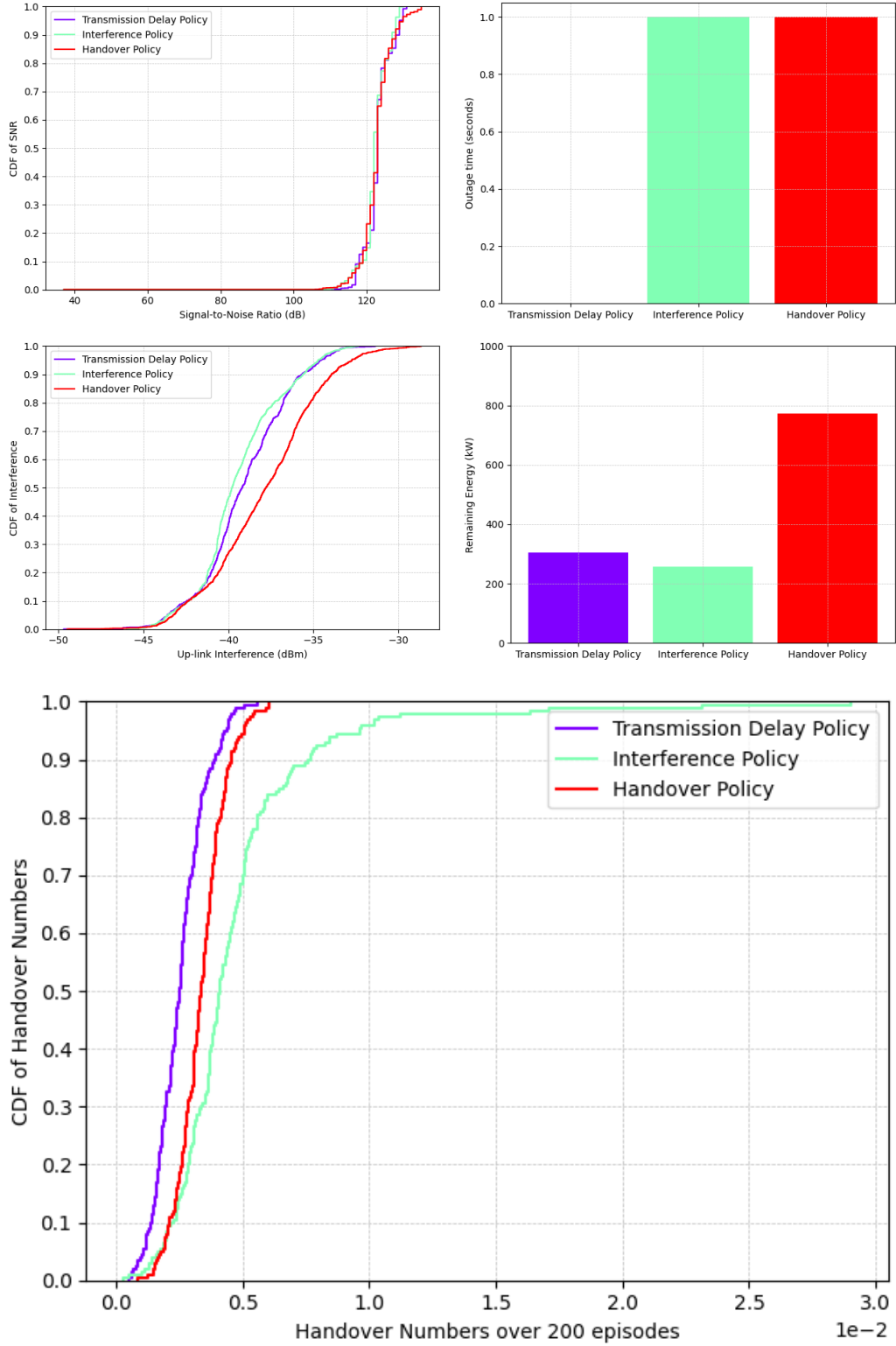


Figure 8: Comparison of SNR, outage time, up-link interference, remaining energy, and number of handovers between our policies, from top left to bottom right

its acceleration actions while the data dumps were indirectly penalized through the transmission delays and minimum RSS requirements. Meanwhile, the data penalization would overwhelm the reward function as soon as a packet was dropped. Both aspects were removed from the current implementation. A more positive integration found during testing is the penalty constraint for a maximum velocity value. Since there was no punishment at the time for travelling at unsafe speeds, the UAV would just attempt to fly as quickly as possible in the shortest amount of time while exhausting all its energy reserves in the process. This integration has effectively discouraged the agent from learning that behaviour.

8. Conclusions and Future Work

The main objective of our work was the development of a realistic environment for an accurate optimization of the trajectories and handover actions of a UAV agent. Most studies on this subject address either the itineraries or the handoffs of the drone, neglecting the possible advantages in formulating a joint solution for a problem with a common root: the drone's inherent flight paths at high-altitude speeds. Furthermore, many of their simulations employ abstractions that do not adjust well to the nature of aerial communication channels and standardized networking protocols. Thus, we have utilized a deterministic raytracing simulation in a real-world 3D model of Barcelona in order to implement our environment with a real-world BS distribution, realistic handover triggers, and accurate resource allocation. Once the environment infrastructure was complete, we formulated an RL problem for a PPO model to solve. After extensive experimentation, we have discovered that our RL agent has the ability to traverse the map and optimize its path in order to obtain the best possible metrics at any given moment. However, the UAV's design impedes it from arriving to its destination in an adequate manner, leading to what seems like wandering behaviours. Our solution prioritizes persistent connectivity over task completion.

Our findings uncover many future research avenues for the joint optimization of trajectory planning and handover management of UAVs. For instance, we could attempt to use stochastic data to assess the effectiveness of the model instead of the deterministic data used in this study. During the development of the thesis, this question was proposed due to the large number of papers employing stochastic simulations for their performance evaluations. Since this research item was deemed to be out of the scope of our project, we leave these comparisons for future work. Besides the design of the simulations, we could also test alternative formulations for the reward function. The trajectory planning aspect of this work may improve through innovative rewards computations. Another option is to extend the training sessions, at the risk of overfitting. Either way, our work serves as a starting point for the implementation of realistic environments for the joint handover and trajectory optimization.

References

- [1] International Civil Aviation Organization (ICAO), *Unmanned aircraft systems traffic management (UTM): A common framework with core principles for global harmonization*, 4th ed. <https://www.icao.int/sites/default/files/left-menu-pdfs/UTM%20Framework%20Edition%204.pdf>.
- [2] E. Vinogradov, H. Sallouha, S. De Bast, M. M. Azari, and S. Pollin, "Tutorial on uavs: A blue sky view on wireless communication", *Journal of Mobile Multimedia*, vol. 14, no. 4, pp. 395–468, 2018. DOI: [10.13052/jmm1550-4646.1443](https://doi.org/10.13052/jmm1550-4646.1443).
- [3] A. Colpaert, E. Vinogradov, and S. Pollin, "3d beamforming and handover analysis for uav networks", in *2020 IEEE Globecom Workshops (GC Wkshps)*, 2020, pp. 1–6. DOI: [10.1109/GCWkshps50303.2020.9367570](https://doi.org/10.1109/GCWkshps50303.2020.9367570).
- [4] J. Tanveer, A. Haider Malik, R. Ali, and A. Kim, "Reinforcement learning-based optimization for drone mobility in 5g and beyond ultra-dense networks", *Computers, Materials & Continua*, vol. 68, pp. 3807–3823, Mar. 2021. DOI: [10.32604/cmc.2021.016087](https://doi.org/10.32604/cmc.2021.016087).
- [5] Y. Jang, S. M. Raza, M. Kim, and H. Choo, "Proactive handover decision for uavs with deep reinforcement learning", *Sensors*, vol. 22, no. 3, 2022, ISSN: 1424-8220. DOI: [10.3390/s22031200](https://doi.org/10.3390/s22031200). [Online]. Available: <https://www.mdpi.com/1424-8220/22/3/1200>.
- [6] J. Zhong, L. Zhang, M. Alhabo, J. Serugunda, and S. N. Mugala, "A hybrid scheme using topsis and q-learning for handover decision making in uav assisted heterogeneous network", *IEEE Access*, vol. 12, pp. 31 422–31 430, 2024. DOI: [10.1109/ACCESS.2024.3368916](https://doi.org/10.1109/ACCESS.2024.3368916).
- [7] I. A. Meer et al., *Explainable ai for uav mobility management: A deep q-network approach for handover minimization*, 2025. arXiv: [2504.18371](https://arxiv.org/abs/2504.18371) [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2504.18371>.
- [8] M. Benzaghta, S. Ammar, D. López-Pérez, B. Shihada, and G. Geraci, *Data-driven design of 3gpp handover parameters with bayesian optimization and transfer learning*, 2025. arXiv: [2504.02633](https://arxiv.org/abs/2504.02633) [cs.IT]. [Online]. Available: <https://arxiv.org/abs/2504.02633>.
- [9] X. Liu, Y. Liu, Y. Chen, and L. Hanzo, "Trajectory design and power control for multi-uav assisted wireless networks: A machine learning approach", *IEEE Transactions on Vehicular Technology*, vol. 68, no. 8, pp. 7957–7969, 2019. DOI: [10.1109/TVT.2019.2920284](https://doi.org/10.1109/TVT.2019.2920284).
- [10] L. Wang, K. Wang, C. Pan, W. Xu, N. Aslam, and L. Hanzo, "Multi-agent deep reinforcement learning-based trajectory planning for multi-uav assisted mobile edge computing", *IEEE Transactions on Cognitive Communications and Networking*, vol. 7, no. 1, pp. 73–84, 2021. DOI: [10.1109/TCCN.2020.3027695](https://doi.org/10.1109/TCCN.2020.3027695).
- [11] L. Wang, K. Wang, C. Pan, W. Xu, N. Aslam, and A. Nallanathan, "Deep reinforcement learning based dynamic trajectory control for uav-assisted mobile edge computing", *IEEE Transactions on Mobile Computing*, vol. 21, no. 10, pp. 3536–3550, 2022. DOI: [10.1109/TMC.2021.3059691](https://doi.org/10.1109/TMC.2021.3059691).
- [12] B. Zhu, E. Bedeer, H. H. Nguyen, R. Barton, and J. Henry, "Uav trajectory planning in wireless sensor networks for energy consumption minimization by deep reinforcement learning", *IEEE Transactions on Vehicular Technology*, vol. 70, no. 9, pp. 9540–9554, 2021. DOI: [10.1109/TVT.2021.3102161](https://doi.org/10.1109/TVT.2021.3102161).
- [13] A. Hajijamali Arani, M. M. Azari, P. Hu, Y. Zhu, H. Yanikomeroglu, and S. Safavi-Naeini, "Reinforcement learning for energy-efficient trajectory design of uavs", *IEEE Internet of Things Journal*, vol. 9, no. 11, pp. 9060–9070, 2022. DOI: [10.1109/JIOT.2021.3118322](https://doi.org/10.1109/JIOT.2021.3118322).
- [14] W. Lee, Y. Jeon, T. Kim, and Y.-I. Kim, "Deep reinforcement learning for uav trajectory design considering mobile ground users", *Sensors*, vol. 21, no. 24, 2021, ISSN: 1424-8220. DOI: [10.3390/s21248239](https://doi.org/10.3390/s21248239). [Online]. Available: <https://www.mdpi.com/1424-8220/21/24/8239>.
- [15] A. B. Bhandarkar and S. K. Jayaweera, "Optimal trajectory learning for uav-mounted mobile base stations using rl and greedy algorithms", in *2021 17th International Conference on Wireless and Mobile Computing, Networking and Communications (WiMob)*, 2021, pp. 13–18. DOI: [10.1109/WiMob52687.2021.9606384](https://doi.org/10.1109/WiMob52687.2021.9606384).

- [16] Z. Chang, H. Deng, L. You, G. Min, S. Garg, and G. Kaddoum, "Trajectory design and resource allocation for multi-uav networks: Deep reinforcement learning approaches", *IEEE Transactions on Network Science and Engineering*, vol. 10, no. 5, pp. 2940–2951, 2023. DOI: [10.1109/TNSE.2022.3171600](https://doi.org/10.1109/TNSE.2022.3171600).
- [17] N. Cherif, W. Jaafar, H. Yanikomeroglu, and A. Yongacoglu, "RI-based cargo-uav trajectory planning and cell association for minimum handoffs, disconnectivity, and energy consumption", *IEEE Transactions on Vehicular Technology*, vol. 73, no. 5, pp. 7304–7309, 2024. DOI: [10.1109/TVT.2023.3340177](https://doi.org/10.1109/TVT.2023.3340177).
- [18] Y. Deng, I. A. Meer, S. Zhang, M. Ozger, and C. Cavdar, "D3qn-based trajectory and handover management for uavs co-existing with terrestrial users", in *2023 21st International Symposium on Modeling and Optimization in Mobile, Ad Hoc, and Wireless Networks (WiOpt)*, 2023, pp. 103–110. DOI: [10.23919/WiOpt58741.2023.10349832](https://doi.org/10.23919/WiOpt58741.2023.10349832).
- [19] H.-S. Im, K.-Y. Kim, J. Chung, and S.-H. Lee, "Joint optimization of trajectory and handover strategy for cellular-enabled uav", *IEEE Access*, vol. 13, pp. 136 970–136 984, 2025. DOI: [10.1109/ACCESS.2025.3595184](https://doi.org/10.1109/ACCESS.2025.3595184).
- [20] Y. Deng, S. Zhang, I. A. Meer, M. Ozger, and C. Cavdar, "Joint trajectory and handover management for uavs co-existing with terrestrial users: A multi-agent drl approach", *IEEE Transactions on Cognitive Communications and Networking*, pp. 1–1, 2025. DOI: [10.1109/TCCN.2025.3578506](https://doi.org/10.1109/TCCN.2025.3578506).
- [21] S. Mahboob and L. Liu, "Revolutionizing future connectivity: A contemporary survey on ai-empowered satellite-based non-terrestrial networks in 6g", *IEEE Communications Surveys & Tutorials*, vol. 26, no. 2, pp. 1279–1321, 2024. DOI: [10.1109/COMST.2023.3347145](https://doi.org/10.1109/COMST.2023.3347145).
- [22] A. Iqbal et al., "Empowering non-terrestrial networks with artificial intelligence: A survey", *IEEE Access*, vol. 11, pp. 100 986–101 006, 2023. DOI: [10.1109/ACCESS.2023.3314732](https://doi.org/10.1109/ACCESS.2023.3314732).
- [23] T. Cattai et al., "Multi-uav reinforcement learning with realistic communication models: Recent advances and challenges", *IEEE Open Journal of Vehicular Technology*, vol. 6, pp. 2067–2081, 2025. DOI: [10.1109/OJVT.2025.3586774](https://doi.org/10.1109/OJVT.2025.3586774).
- [24] I. Shayeia et al., "Handover management for drones in future mobile networks—a survey", *Sensors*, vol. 22, no. 17, 2022, ISSN: 1424-8220. DOI: [10.3390/s22176424](https://doi.org/10.3390/s22176424). [Online]. Available: <https://www.mdpi.com/1424-8220/22/17/6424>.
- [25] J. Tanveer, A. Haider, R. Ali, and A. Kim, "An overview of reinforcement learning algorithms for handover management in 5g ultra-dense small cell networks", *Applied Sciences*, vol. 12, no. 1, 2022, ISSN: 2076-3417. DOI: [10.3390/app12010426](https://doi.org/10.3390/app12010426). [Online]. Available: <https://www.mdpi.com/2076-3417/12/1/426>.
- [26] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, *Proximal policy optimization algorithms*, 2017. arXiv: [1707.06347](https://arxiv.org/abs/1707.06347) [cs.LG]. [Online]. Available: <https://arxiv.org/abs/1707.06347>.
- [27] Y. Che et al., "3d-globfp: The first global three-dimensional building footprint dataset", *Earth System Science Data*, vol. 16, no. 11, pp. 5357–5374, 2024. DOI: [10.5194/essd-16-5357-2024](https://doi.org/10.5194/essd-16-5357-2024). [Online]. Available: <https://essd.copernicus.org/articles/16/5357/2024/>.
- [28] Ministerio para la Transformación Digital y de la Función Pública, *Geoportal de Radioaficionados y Estaciones de Telecomunicaciones (Infoantenas)*, <https://geoportal.minetur.gob.es/VCTEL/vcne.do>, Accessed: 2024-05-20, 2024.
- [29] E. Vinogradov, A. Fakhreddine, A. Saboor, S. Abadal, and S. Pollin, "Spatially consistent air-to-ground channel modeling and simulation via 3d shadow projections", in *2026 International Conference on Computing, Networking and Communications (ICNC)*, 2026, pp. 1–6. DOI: [10.1109/ICNC68183.2026.11416865](https://doi.org/10.1109/ICNC68183.2026.11416865).
- [30] A. Saboor, Z. Cui, A. Colpaert, E. Vinogradov, and S. Pollin, *Cash: Context-aware smart handover for reliable uav connectivity on aerial corridors*, Aug. 2025. DOI: [10.48550/arXiv.2508.03862](https://doi.org/10.48550/arXiv.2508.03862).
- [31] R. D'Andrea, "Guest editorial can drones deliver?", *IEEE Transactions on Automation Science and Engineering*, vol. 11, no. 3, pp. 647–648, 2014. DOI: [10.1109/TASE.2014.2326952](https://doi.org/10.1109/TASE.2014.2326952).
- [32] Y. Huang, G. Wei, and Y. Wang, "V-d d3qn: The variant of double deep q-learning network with dueling architecture", in *2018 37th Chinese Control Conference (CCC)*, 2018, pp. 9130–9135. DOI: [10.23919/ChiCC.2018.8483478](https://doi.org/10.23919/ChiCC.2018.8483478).

- [33] H. V. Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double q-learning", ArXiv, vol. abs/1509.06461, 2015. [Online]. Available: <https://api.semanticscholar.org/CorpusID:6208256>.
- [34] Z. Wang, N. de Freitas, and M. Lanctot, "Dueling network architectures for deep reinforcement learning", CoRR, vol. abs/1511.06581, 2015. arXiv: 1511.06581. [Online]. Available: <http://arxiv.org/abs/1511.06581>.
- [35] O. M. Andrychowicz et al., "Learning dexterous in-hand manipulation", *Int. J. Rob. Res.*, vol. 39, no. 1, 3–20, Jan. 2020, ISSN: 0278-3649. DOI: 10.1177/0278364919887447. [Online]. Available: <https://doi.org/10.1177/0278364919887447>.
- [36] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations", *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. [Online]. Available: <http://jmlr.org/papers/v22/20-1364.html>.
- [37] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation", Jun. 2015. DOI: 10.48550/arXiv.1506.02438.

A. List of Abbreviations

Abbreviation	Definition
3GPP	3rd Generation Partnership Project
5G	5th Generation
6G	6th Generation
AI	Artificial Intelligence
BS	Base Station
D3QN	Dueling Double Deep Q-Network
DDPG	Deep Deterministic Policy Gradient
DRL	Deep Reinforcement Learning
DQN	Deep Q-Network
DRQN	Deep Recurrent Q-Networks
EE	Energy Efficiency
GAE	Generalized Advantage Estimation
HO	Handover
KL	Kullback–Leibler divergence
LoS	Line of Sight
LAE	Low-Altitude Economy
MAB	Multi-Armed Bandit
MDP	Markov Decision Process
ML	Machine Learning
NTN	Non-Terrestrial Networks
PPO	Proximal Policy Optimization
QoS	Quality of Service
RF	Radio Frequency
RSSI	Radio Signal Strength Indicator
RSS	Radio Signal Strength
RSRP	Reference Signal Received Power
RL	Reinforcement Learning
RBG	Resource Block Group
SINR	Signal-to-Interference-plus-Noise Ratio
STL	Standard Tessellation Language
TOPSIS	Technique for Order of Preference by Similarity
TTT	Time To Trigger
UAS	Unmanned Aerial System
UAV	Unmanned Aerial Vehicle
UL	Unsupervised Learning
UE	User Equipment

Table 5: List of abbreviations

B. Handover Visualizations

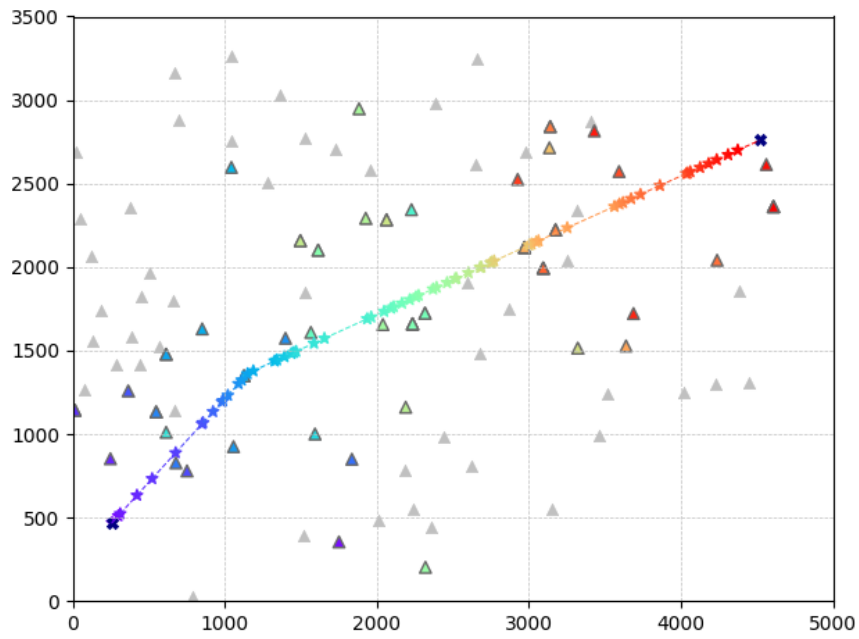


Figure 9: Handover visualization from greedy benchmark

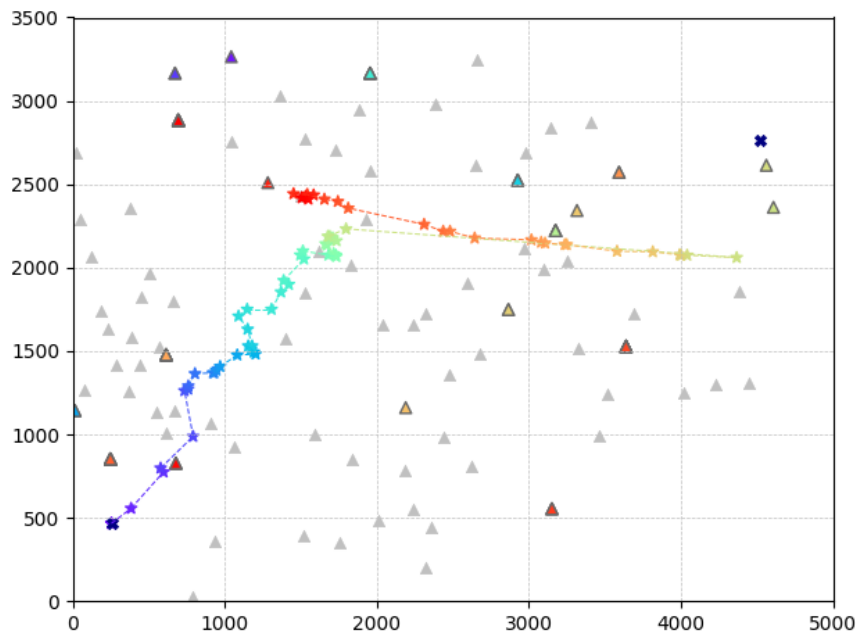


Figure 10: Handover visualization from PPO